

Teach Module 37: Oracle Database Fundamentals

This README captures the Module 37 teaching notes and practice lab. The course document was used only to identify the module topic; the teaching content below is written independently.

Module Goal

Module 37 focuses on the database fundamentals a Java software engineer needs before integrating applications with Oracle:

- Tables, rows, and columns
- Primary keys and foreign keys
- Relationships between tables
- Constraints
- Schema design principles
- Oracle SQL data types
- Indexing basics
- Java connectivity concepts

1. What Is a Relational Database?

A relational database stores data in tables.

Example tables for an employee system:

```
EMPLOYEES
- employee_id
- first_name
- last_name
- department_id
- salary

DEPARTMENTS
- department_id
- department_name
```

Each table represents one type of thing: employees, departments, orders, customers, products, tasks, and so on.

Each row is one record.

Each column is one attribute.

Example:

employee_id	first_name	department_id
1	Priya	10
2	Marcus	20

This means there are two employee records.

2. Primary Keys

A primary key uniquely identifies each row in a table.

```
CREATE TABLE departments (  
    department_id NUMBER PRIMARY KEY,  
    department_name VARCHAR2(100) NOT NULL  
);
```

Here, `department_id` is the unique identifier.

A good primary key should be:

- Unique
- Stable
- Not null
- Simple to reference from other tables

In Oracle, common primary key types are usually `NUMBER` columns, often generated automatically.

3. Foreign Keys

A foreign key connects one table to another.

```
CREATE TABLE employees (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50) NOT NULL,  
    last_name VARCHAR2(50) NOT NULL,  
    department_id NUMBER,  
    CONSTRAINT fk_employee_department  
        FOREIGN KEY (department_id)  
        REFERENCES departments(department_id)  
);
```

This means every `employees.department_id` must match an existing `departments.department_id`.

That relationship protects your data from becoming inconsistent.

For example, Oracle will prevent this if department `999` does not exist:

```
INSERT INTO employees (  
    employee_id, first_name, last_name, department_id  
)  
VALUES (  
    1, 'Ava', 'Shah', 999  
);
```

4. Relationships

One-to-Many

One department has many employees.

```
DEPARTMENTS 1 ---- many EMPLOYEES
```

This is the most common relationship.

One-to-One

One user has one profile.

```
USERS 1 ---- 1 USER_PROFILES
```

This is less common, but useful when splitting sensitive or optional data.

Many-to-Many

Students can enroll in many courses, and courses can have many students.

Relational databases model this using a join table:

```
STUDENTS  
COURSES  
ENROLLMENTS
```

Example:

```
CREATE TABLE enrollments (  
  student_id NUMBER,  
  course_id NUMBER,  
  enrollment_date DATE,  
  PRIMARY KEY (student_id, course_id)  
);
```

5. Constraints

Constraints are rules the database enforces.

Common constraints:

```
NOT NULL
```

Requires a value.

```
UNIQUE
```

Prevents duplicates.

```
PRIMARY KEY
```

Uniquely identifies a row.

FOREIGN KEY

Maintains relationships between tables.

CHECK

Validates a condition.

Example:

```
CREATE TABLE accounts (  
  account_id NUMBER PRIMARY KEY,  
  balance NUMBER(10, 2) CHECK (balance >= 0),  
  email VARCHAR2(255) UNIQUE NOT NULL  
);
```

This prevents negative balances and duplicate emails.

6. Basic Schema Design

A schema is the structure of your database: tables, columns, relationships, constraints, indexes, and other objects.

A good schema usually follows these principles:

- Store each concept in its own table
- Avoid duplicate data
- Use primary keys for identity
- Use foreign keys for relationships
- Choose appropriate data types
- Add constraints to protect correctness
- Design for how the application will query the data

Bad design example:

```
ORDERS  
- order_id  
- customer_name  
- customer_email  
- customer_address  
- product_1  
- product_2  
- product_3
```

Better design:

```
CUSTOMERS  
ORDERS  
PRODUCTS  
ORDER_ITEMS
```

That design is more flexible because one order can have any number of products.

7. Normalization

Normalization means organizing data to reduce duplication and avoid update problems.

Example of duplication:

employee_id	employee_name	department_name
1	Nina	Finance
2	Omar	Finance
3	Leah	Finance

If Finance changes to Corporate Finance, you must update many rows.

Better design:

```
DEPARTMENTS
department_id | department_name
10            | Finance

EMPLOYEES
employee_id | employee_name | department_id
1           | Nina          | 10
2           | Omar          | 10
3           | Leah          | 10
```

Now the department name lives in one place.

8. Oracle SQL Data Types

Important Oracle data types:

NUMBER

Used for integers and decimals.

VARCHAR2(size)

Variable-length text.

CHAR(size)

Fixed-length text. Less commonly used.

DATE

Stores date and time to seconds.

TIMESTAMP

Stores date and time with fractional seconds.

CLOB

Large text data.

BLOB

Binary data, such as files or images.

Example table:

```
CREATE TABLE products (  
  product_id NUMBER PRIMARY KEY,  
  name VARCHAR2(100) NOT NULL,  
  price NUMBER(10, 2) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

`NUMBER(10, 2)` means up to 10 digits total, with 2 digits after the decimal.

Example values:

```
1299.99  
45.50  
1000000.00
```

9. Indexing Basics

An index helps the database find rows faster.

Think of an index like the index at the back of a book. Without it, Oracle may scan many rows. With it, Oracle can jump closer to the right data.

Example:

```
CREATE INDEX idx_employees_last_name  
ON employees(last_name);
```

This can help queries like:

```
SELECT *  
FROM employees  
WHERE last_name = 'Patel';
```

Indexes are not free.

They speed up reads, but they can slow down writes because Oracle must update the index when rows are inserted, updated, or deleted.

Use indexes on columns that are commonly used in:

```
WHERE  
JOIN  
ORDER BY  
GROUP BY
```

Do not index every column automatically.

10. Connecting Java to Oracle

Java applications usually connect to Oracle using JDBC directly or through frameworks like Spring Data JPA.

At the lowest level, JDBC uses a database URL, username, password, and driver.

Example JDBC-style connection:

```
String url = "jdbc:oracle:thin:@localhost:1521/XEPDB1";  
String username = "app_user";  
String password = "secret";  
  
Connection connection = DriverManager.getConnection(url, username, password);
```

A simple query:

```
String sql = "SELECT employee_id, first_name, last_name FROM employees";  
  
try (  
    Connection connection = DriverManager.getConnection(url, username, password);  
    PreparedStatement statement = connection.prepareStatement(sql);  
    ResultSet resultSet = statement.executeQuery()  
) {  
    while (resultSet.next()) {  
        long id = resultSet.getLong("employee_id");  
        String firstName = resultSet.getString("first_name");  
        String lastName = resultSet.getString("last_name");  
  
        System.out.println(id + ": " + firstName + " " + lastName);  
    }  
}
```

In real enterprise apps, you usually avoid raw JDBC for most business logic and use Spring Boot with repositories, connection pooling, and transactions. But knowing what JDBC does underneath is still important.

Quick Check

Answer these before moving to the lab:

1. What is the difference between a primary key and a foreign key?
2. Why should duplicate customer information usually be moved into a separate `customers` table?
3. When might an index help?
4. Why should you not index every column?
5. What Oracle type would you use for money-like values such as `129.99` ?

Practice Exercises

Exercise 1: Design Tables

Design tables for a small library system.

Entities:

- Members
- Books
- Loans

Requirements:

- A member can borrow many books.
- A book can be loaned many times over its lifetime.
- Each loan tracks borrow date, due date, and return date.

Create tables with:

- Primary keys
- Foreign keys
- `NOT NULL` constraints
- Reasonable Oracle data types

Exercise 2: Create a Customer Orders Schema

Build a schema for:

- Customers
- Orders
- Products
- Order items

Practice the classic relationship:

```
CUSTOMERS 1 -> many ORDERS
ORDERS 1 -> many ORDER_ITEMS
PRODUCTS 1 -> many ORDER_ITEMS
```

Add constraints so:

- Customer email is unique
- Product price cannot be negative
- Order status must be one of `NEW` , `PAID` , `SHIPPED` , `CANCELLED`

Exercise 3: Practice Data Types

Create a `products` table using Oracle types:

```
product_id
name
description
price
stock_quantity
created_at
image_data
```

Choose appropriate types from:

```
NUMBER
VARCHAR2
CLOB
BLOB
DATE
TIMESTAMP
```

Then explain why you chose each type.

Exercise 4: Add Constraints

Create an `employees` table with these rules:

- `employee_id` is the primary key
- `email` must be unique
- `first_name` and `last_name` are required
- `salary` must be greater than 0
- `department_id` references a `departments` table

Use:

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
```

Exercise 5: Normalize a Bad Table

Start with this bad table:

```
ORDERS_BAD
- order_id
- customer_name
- customer_email
- customer_address
- product_1_name
- product_1_price
- product_2_name
- product_2_price
```

- product_3_name
- product_3_price

Redesign it into normalized tables.

Expected tables:

```
CUSTOMERS
ORDERS
PRODUCTS
ORDER_ITEMS
```

Exercise 6: Create Indexes

For a `tasks` table:

```
tasks (
  task_id,
  project_id,
  assigned_user_id,
  status,
  due_date,
  title
)
```

Decide which columns should be indexed for these queries:

```
SELECT * FROM tasks WHERE project_id = 10;

SELECT * FROM tasks WHERE assigned_user_id = 5;

SELECT * FROM tasks WHERE status = 'IN_PROGRESS';

SELECT * FROM tasks WHERE due_date < SYSDATE;
```

Then write the `CREATE INDEX` statements.

Exercise 7: Insert Valid and Invalid Data

Create tables with constraints, then try inserting both valid and invalid records.

Example invalid cases:

- Insert an employee without a last name
- Insert duplicate emails
- Insert a task with status `WAITING`, if only `TODOT`, `IN_PROGRESS`, and `DONE` are allowed
- Insert an order item referencing a product that does not exist

The goal is to see how the database protects data integrity.

Exercise 8: Draw an ERD

Draw an entity relationship diagram for a banking app.

Entities:

- Customers
- Accounts
- Transactions

Rules:

- One customer can have many accounts
- One account can have many transactions
- A transaction has an amount, transaction type, and timestamp
- Transaction type should be constrained to allowed values

Exercise 9: Java Connection Practice

Write a small Java program that connects to Oracle and runs:

```
SELECT employee_id, first_name, last_name FROM employees
```

Practice using:

```
Connection  
PreparedStatement  
ResultSet
```

Also practice closing resources with `try-with-resources`.

Exercise 10: Mini Project

Design a database for a task management app.

Required tables:

```
USERS  
PROJECTS  
TASKS  
COMMENTS
```

Rules:

- A user can own many projects
- A project can have many tasks
- A task can have many comments
- A comment belongs to one user
- Task status must be controlled with a `CHECK` constraint
- Email must be unique
- Add indexes for common lookup columns

Lab: Oracle Database Design

Scenario

You are building the database for a small Task Management System used by software teams.

The system needs to track:

- Users
- Projects
- Tasks
- Comments

A user can own many projects. A project can have many tasks. A task can be assigned to a user. A task can have many comments.

Lab Goals

By the end, you should be able to:

- Design relational tables
- Use primary keys and foreign keys
- Apply `NOT NULL`, `UNIQUE`, and `CHECK` constraints
- Choose Oracle SQL data types
- Create useful indexes
- Insert and query sample data

Part 1: Create Tables

Create the `users` table:

```
CREATE TABLE users (  
    user_id NUMBER PRIMARY KEY,  
    full_name VARCHAR2(100) NOT NULL,  
    email VARCHAR2(255) UNIQUE NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL  
);
```

Create the `projects` table:

```
CREATE TABLE projects (  
    project_id NUMBER PRIMARY KEY,  
    owner_user_id NUMBER NOT NULL,  
    project_name VARCHAR2(150) NOT NULL,  
    description VARCHAR2(500),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,  
  
    CONSTRAINT fk_project_owner  
        FOREIGN KEY (owner_user_id)  
        REFERENCES users(user_id)  
);
```

Create the `tasks` table:

```
CREATE TABLE tasks (  
    task_id NUMBER PRIMARY KEY,  
    project_id NUMBER NOT NULL,
```

```

assigned_user_id NUMBER,
title VARCHAR2(200) NOT NULL,
description VARCHAR2(1000),
status VARCHAR2(20) DEFAULT 'TODO' NOT NULL,
priority VARCHAR2(20) DEFAULT 'MEDIUM' NOT NULL,
due_date DATE,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,

CONSTRAINT fk_task_project
    FOREIGN KEY (project_id)
    REFERENCES projects(project_id),

CONSTRAINT fk_task_assignee
    FOREIGN KEY (assigned_user_id)
    REFERENCES users(user_id),

CONSTRAINT chk_task_status
    CHECK (status IN ('TODO', 'IN_PROGRESS', 'DONE', 'BLOCKED')),

CONSTRAINT chk_task_priority
    CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH'))
);

```

Create the `comments` table:

```

CREATE TABLE comments (
    comment_id NUMBER PRIMARY KEY,
    task_id NUMBER NOT NULL,
    user_id NUMBER NOT NULL,
    comment_text VARCHAR2(1000) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,

    CONSTRAINT fk_comment_task
        FOREIGN KEY (task_id)
        REFERENCES tasks(task_id),

    CONSTRAINT fk_comment_user
        FOREIGN KEY (user_id)
        REFERENCES users(user_id)
);

```

Part 2: Create Indexes

```

CREATE INDEX idx_projects_owner_user_id
ON projects(owner_user_id);

CREATE INDEX idx_tasks_project_id
ON tasks(project_id);

CREATE INDEX idx_tasks_assigned_user_id

```

```
ON tasks(assigned_user_id);

CREATE INDEX idx_tasks_status
ON tasks(status);

CREATE INDEX idx_comments_task_id
ON comments(task_id);
```

Part 3: Insert Sample Data

```
INSERT INTO users (user_id, full_name, email)
VALUES (1, 'Ava Patel', 'ava.patel@example.com');

INSERT INTO users (user_id, full_name, email)
VALUES (2, 'Marcus Lee', 'marcus.lee@example.com');

INSERT INTO projects (project_id, owner_user_id, project_name, description)
VALUES (100, 1, 'Inventory API', 'Backend service for inventory tracking');

INSERT INTO tasks (
    task_id, project_id, assigned_user_id, title, description, status, priority, due_date
)
VALUES (
    1000, 100, 2, 'Create product table', 'Design product schema and constraints',
    'TODO', 'HIGH', DATE '2026-08-15'
);

INSERT INTO tasks (
    task_id, project_id, assigned_user_id, title, description, status, priority
)
VALUES (
    1001, 100, 1, 'Define indexes', 'Add indexes for common queries',
    'IN_PROGRESS', 'MEDIUM'
);

INSERT INTO comments (comment_id, task_id, user_id, comment_text)
VALUES (5000, 1000, 1, 'Make sure price uses NUMBER with decimal precision.');
```

```
COMMIT;
```

Part 4: Query the Data

Find all tasks for a project:

```
SELECT task_id, title, status, priority
FROM tasks
WHERE project_id = 100;
```

Find tasks assigned to a user:

```
SELECT task_id, title, status
FROM tasks
WHERE assigned_user_id = 2;
```

Join tasks with project and user details:

```
SELECT
    p.project_name,
    t.title,
    t.status,
    u.full_name AS assigned_to
FROM tasks t
JOIN projects p
    ON t.project_id = p.project_id
LEFT JOIN users u
    ON t.assigned_user_id = u.user_id;
```

Find comments for a task:

```
SELECT
    t.title,
    u.full_name,
    c.comment_text,
    c.created_at
FROM comments c
JOIN tasks t
    ON c.task_id = t.task_id
JOIN users u
    ON c.user_id = u.user_id
WHERE c.task_id = 1000;
```

Part 5: Test Constraints

Try inserting an invalid task status:

```
INSERT INTO tasks (
    task_id, project_id, assigned_user_id, title, status
)
VALUES (
    1002, 100, 1, 'Invalid task', 'WAITING'
);
```

Expected result: Oracle should reject it because `WAITING` is not allowed by the `chk_task_status` constraint.

Try inserting a duplicate email:

```
INSERT INTO users (user_id, full_name, email)
VALUES (3, 'Ava Duplicate', 'ava.patel@example.com');
```

Expected result: Oracle should reject it because `email` must be unique.

Try inserting a task for a project that does not exist:

```
INSERT INTO tasks (  
    task_id, project_id, assigned_user_id, title  
)  
VALUES (  
    1003, 999, 1, 'Missing project test'  
);
```

Expected result: Oracle should reject it because project `999` does not exist.

Part 6: Challenge

Extend the design by adding a `task_history` table.

It should track:

- History ID
- Task ID
- Old status
- New status
- Changed by user ID
- Changed timestamp

Suggested start:

```
CREATE TABLE task_history (  
    history_id NUMBER PRIMARY KEY,  
    task_id NUMBER NOT NULL,  
    old_status VARCHAR2(20),  
    new_status VARCHAR2(20) NOT NULL,  
    changed_by_user_id NUMBER NOT NULL,  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,  
  
    CONSTRAINT fk_history_task  
        FOREIGN KEY (task_id)  
        REFERENCES tasks(task_id),  
  
    CONSTRAINT fk_history_changed_by  
        FOREIGN KEY (changed_by_user_id)  
        REFERENCES users(user_id),  
  
    CONSTRAINT chk_history_old_status  
        CHECK (old_status IN ('TODO', 'IN_PROGRESS', 'DONE', 'BLOCKED')),  
  
    CONSTRAINT chk_history_new_status  
        CHECK (new_status IN ('TODO', 'IN_PROGRESS', 'DONE', 'BLOCKED'))  
);
```

Deliverable

Submit:

- Your final `CREATE TABLE` statements
- Your `CREATE INDEX` statements
- At least 5 `INSERT` statements
- At least 3 working `SELECT` queries
- A short explanation of your schema design: which tables relate to which, and why

Main Takeaway

A Java developer does not need to be a full-time database administrator, but you do need to understand tables, keys, constraints, data types, indexes, and connections well enough to design clean persistence and avoid painful application bugs.